Stack Tecnológico del Sistema de Información y Control del Transmetro

Luis Orozco

Carné 3590 22 5339

Universidad Mariano Gálvez de Guatemala

Facultad de Ingeniería en Sistemas

Aseguramiento de la Calidad del Software

2 de octubre de 2026

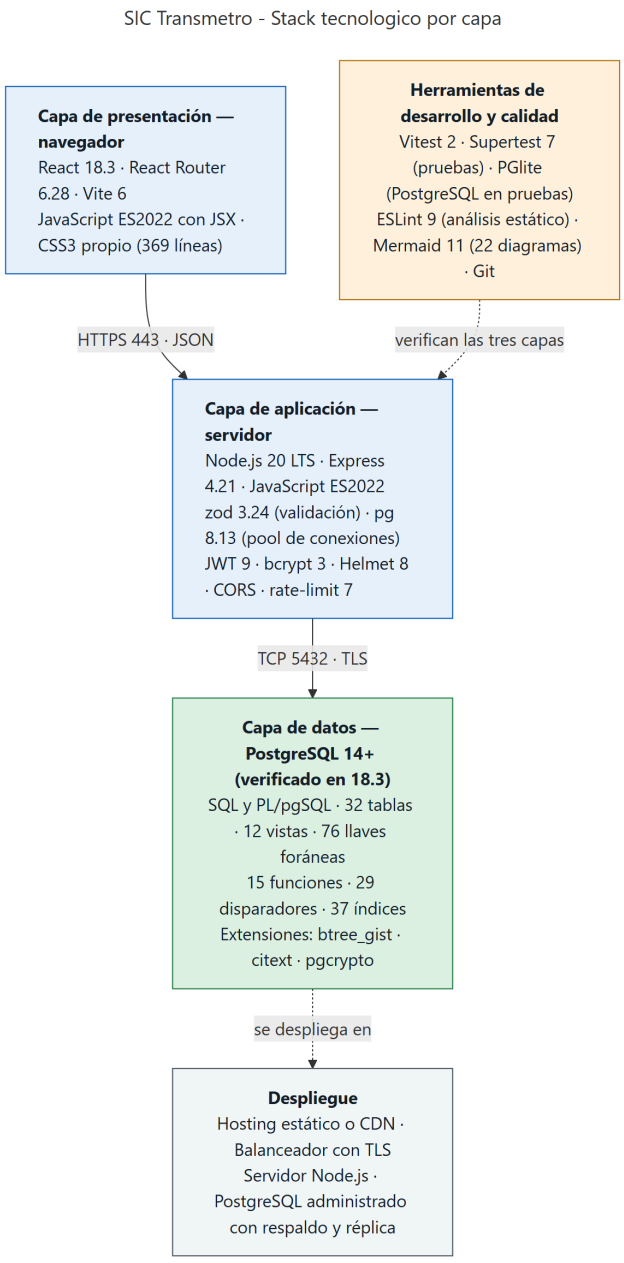
Stack Tecnológico del Sistema de Información y Control del Transmetro

Se denomina stack tecnológico al conjunto de lenguajes, bibliotecas, servicios y herramientas que un sistema utiliza en cada una de sus capas. Este documento presenta el stack completo del Sistema de Información y Control del Transmetro (SIC Transmetro), con la versión exacta de cada componente, su función dentro del sistema y los requisitos de entorno necesarios para ejecutarlo.

El sistema adopta una arquitectura en tres capas desacopladas. Una característica distintiva de su stack es que emplea un solo lenguaje de programación de aplicación, JavaScript, tanto en el navegador como en el servidor, con SQL y PL/pgSQL reservados para la capa de datos.

Figura 1

Stack tecnológico del sistema organizado por capa



Nota. Las flechas continuas indican comunicación en tiempo de ejecución y las punteadas, relaciones de verificación y despliegue. Elaboración propia.

Capa de Presentación

Comprende todo lo que se ejecuta en el navegador del usuario, sea la computadora de una estación o un dispositivo móvil.

Tabla 1

Componentes de la capa de presentación

Componente	**Versión**	**Función en el sistema**
React	18.3.1	Construcción de la interfaz mediante componentes reutilizables; incluye el componente que oculta acciones según el permiso del usuario
React DOM	18.3.1	Renderizado de los componentes en el navegador
React Router DOM	6.28.0	Enrutamiento de las once pantallas y protección de rutas por módulo y nivel de permiso
Vite	6.0.7	Servidor de desarrollo con recarga inmediata y empaquetado para producción
Plugin de React para Vite	4.3.4	Transformación de la sintaxis JSX
CSS3 propio	Estándar vigente	Hoja de estilos de 369 líneas con variables, flexbox y diseño adaptable a partir de 820 píxeles

Nota. No se emplea ninguna biblioteca de componentes ni framework de CSS de terceros.

Capa de Aplicación

Concentra la lógica de autenticación, autorización, validación y orquestación de transacciones.

Tabla 2

Componentes de la capa de aplicación

Componente	**Versión**	**Función en el sistema**
Node.js	20 LTS o superior	Entorno de ejecución de JavaScript en el servidor
Express	4.21.2	Marco de trabajo para la interfaz REST y el encadenamiento de middlewares
pg	8.13.1	Controlador de PostgreSQL con agrupación de conexiones
zod	3.24.1	Validación y conversión de tipos de los datos de entrada
jsonwebtoken	9.0.2	Emisión y verificación de los tokens de acceso y de refresco
bcryptjs	3.0.2	Cifrado de contraseñas con factor de coste 12
Helmet	8.0.0	Cabeceras de seguridad HTTP
CORS	2.8.5	Control de orígenes permitidos
express-rate-limit	7.5.0	Límite de peticiones por dirección, con restricción adicional en el inicio de sesión
morgan	1.10.0	Registro de peticiones
dotenv	16.4.5	Carga de variables de entorno

Nota. Las versiones corresponden a las declaradas en el archivo de dependencias del proyecto.

Capa de Datos

Tabla 3

Componentes de la capa de datos

Componente	Versión	Función en el sistema
PostgreSQL	14 o superior, verificado en 18.3	Motor relacional; almacena los datos y garantiza las reglas de negocio
SQL	SQL:2016, dialecto PostgreSQL	Definición del esquema, consultas y vistas
PL/pgSQL	Nativo del motor	15 funciones y 29 disparadores de reglas de negocio y auditoría
btree_gist	Extensión del motor	Habilita las restricciones de exclusión que combinan igualdad con rangos de tiempo
citext	Extensión del motor	Comparación de usuario y correo sin distinguir mayúsculas
pgcrypto	Extensión del motor	Funciones de hash para el número de tarjeta

Nota. El esquema contiene 32 tablas, 12 vistas, 76 llaves foráneas, 47 restricciones CHECK, 28 restricciones UNIQUE y 4 restricciones EXCLUDE.

Herramientas de Desarrollo y Calidad

Estas herramientas no forman parte del producto desplegado, pero son parte del stack porque determinan cómo se construye y se verifica.

Tabla 4

Herramientas de desarrollo y aseguramiento de la calidad

Herramienta	Versión	Función en el proyecto
Vitest	2.1.8	Ejecución de pruebas unitarias y de integración
Supertest	7.0.0	Pruebas de los puntos de acceso REST sin levantar el servidor
PGlite	0.5.8	Ejecución de PostgreSQL dentro de las pruebas, sin instalar un servidor
ESLint	9.17.0	Análisis estático del código
Mermaid CLI	11	Generación de los 22 diagramas a partir de fuentes de texto versionadas
Git	2.51	Control de versiones del código, del script y de los diagramas
npm	10 o superior	Gestión de dependencias y ejecución de guiones

Nota. PGlite permite verificar el script de base de datos en cualquier equipo de desarrollo y, posteriormente, dentro de un proceso de integración continua.

Infraestructura de Despliegue

Tabla 5

Componentes de la infraestructura de despliegue

Nodo	**Tecnología**	**Función**
Hosting estático o red de distribución de contenido	Servicio en la nube	Entrega de los archivos generados por Vite
Balanceador de carga	Servicio en la nube con certificado TLS	Terminación de HTTPS y distribución entre instancias
Servidor de aplicación	Node.js 20 LTS	Ejecución de la interfaz REST; no almacena estado en memoria, por lo que admite varias instancias
Base de datos	PostgreSQL administrado	Almacenamiento con respaldo diario, registro de transacciones continuo y réplica

Nota. La ausencia de estado en memoria del servidor de aplicación es la condición que permite escalar horizontalmente sin sesiones compartidas.

Requisitos del Entorno

Tabla 6

Requisitos para ejecutar el sistema

Requisito	**Versión mínima**	**Ámbito**
Node.js	20	Desarrollo y servidor de producción
npm	10	Desarrollo
PostgreSQL	14	Base de datos, local o administrada
Navegador web	Versión vigente de Chrome, Edge, Firefox o Safari	Cliente
Git	2.40	Desarrollo

Nota. El sistema no requiere instalación de software adicional en los equipos de estación, puesto que la aplicación se ejecuta en el navegador.

Criterios de Selección Aplicados

Tabla 7

Criterios utilizados para seleccionar cada componente del stack

Criterio	**Aplicación en el proyecto**
Cumplimiento del requerimiento del proyecto	El motor de base de datos debía ser relacional y de código abierto, condición que cumple PostgreSQL
Capacidad de garantizar las reglas de negocio	Se seleccionó el único motor libre que ofrece restricciones de exclusión sobre rangos temporales
Reducción de tecnologías a mantener	Un solo lenguaje de aplicación, sin mapeador objeto-relacional y sin framework de CSS
Madurez y soporte	Todos los componentes son de uso extendido, con documentación vigente y versiones con soporte a largo plazo
Costo de licencia	Todos los componentes son de código abierto y sin costo de licencia
Facilidad de mantenimiento por terceros	Se privilegiaron tecnologías con amplia documentación, considerando que el sistema será mantenido por personal distinto al que lo construyó

Nota. Las alternativas evaluadas y descartadas se detallan en el documento de lenguajes, motor de base de datos y métricas.

Puesta en Marcha

La instalación y ejecución del sistema se realiza mediante los guiones siguientes:

```
npm install         Instala las dependencias (servidor y aplicación web)
npm run db:reset    Crea el esquema completo y carga los datos de ejemplo
npm run dev         Inicia la interfaz REST en el puerto 4000
npm test            Ejecuta las pruebas automatizadas
npm run build       Genera los archivos de producción de la aplicación
web
```

Durante el desarrollo, Vite redirige las peticiones dirigidas a la ruta de la interfaz REST hacia el servidor local, lo cual evita configurar políticas de origen cruzado. En producción, el origen permitido se declara mediante una variable de entorno.

Conclusiones

El stack del SIC Transmetro se compone de siete tecnologías en la capa de presentación, once en la capa de aplicación y seis en la capa de datos, además de siete herramientas de desarrollo y aseguramiento de la calidad. Todos los componentes son de código abierto y carecen de costo de licencia.

La decisión de mayor peso dentro del stack corresponde al motor de base de datos. PostgreSQL no se seleccionó por preferencia sino por una capacidad concreta: las restricciones de exclusión sobre rangos temporales, que garantizan los requerimientos RF-04 y RF-06 de forma incondicional. El resto de las decisiones responde al criterio de reducir la cantidad de tecnologías que el equipo debe mantener en paralelo.

Referencias

Meta Platforms. (s.f.). *React documentation*. https://react.dev

OpenJS Foundation. (s.f.). *Node.js documentation*. https://nodejs.org/docs

PostgreSQL Global Development Group. (s.f.). *PostgreSQL documentation*. https://www.postgresql.org/docs/

Pressman, R. S., y Maxim, B. R. (2020). *Software engineering: A practitioner's approach* (9.ª ed.). McGraw-Hill Education.

Sommerville, I. (2016). *Software engineering* (10.ª ed.). Pearson Education.